□ SELF-EVALUATING LESSON CONTENT GENERATOR

A Simple, Plain-Language Project Explainer

Explained across 5 easy-to-follow phases

What Is This Project, In One Breath?

This project is an AI system that writes short lessons on tricky technical topics (like "Retrieval-Augmented Generation", or RAG) for 12th-grade students in India who are not fluent in English. But it does something extra: it does not just write the lesson once and stop. It checks its own work against a strict checklist, and if the lesson fails, it rewrites it — up to 3 times — until it passes, or it gives up and stops.

> ***Think of it like this:*** *a student writes an essay, a strict teacher grades it against a checklist, hands back specific comments ("this sentence is too long," "you forgot an example"), and the student rewrites the essay using those comments. This happens automatically, with no human involved, up to 3 rounds.*

Quick Overview of the 5 Phases

Phase	Name	In One Line
1	Setup & Tools	What the system is built with, and how the code is organised.
2	Rules & Checklist	Who the lesson is for, and the 5 checks every lesson must pass.
3	Write → Check → Rewrite	The core loop: draft the lesson, grade it, fix it, repeat (max 3 times).
4	Memory & Testing	How the system remembers past mistakes, and how it was proven to work.
5	Final Result & Takeaways	What was actually delivered, and the key lessons learned building it.

Phase 1: Setup & Tools — What Is This Built With?

Before writing any lesson, the project needed a solid foundation: a set of tools to think and write, a way to control the flow of steps, and a clean folder structure so nothing gets messy.

The Main Tools Used

- **LLM Engine:** The "brain" that writes and grades — Groq Cloud running an open-weights model called GPT-OSS-120B.
- **Flow Controller:** LangGraph — the tool that controls the step-by-step flow (write, check, rewrite, stop) and remembers what happened at each step.
- **Structure Enforcer:** Pydantic — makes sure the AI's output always comes back in a clean, predictable format (not random free text) so the code can read it reliably.
- **Glue Framework:** LangChain — connects the prompts, the model, and the structured output together smoothly.
- **Safe Key Storage:** A .env file keeps the secret API key separate from the code, so it's never exposed.

How the Code Is Organised

The project is split into clear, labelled folders so each part has one job:

- **config/persona.py** → Holds the rules about who the learner is and how to speak to them.
- **src/graph/nodes.py** → Contains the logic for writing, grading, retrying, and deciding what happens next.
- **src/graph/state.py** → Keeps track of everything happening during a run (the topic, the draft, the score, the retry count).
- **src/schemas/** → Define exactly what a "lesson" and a "grading report" must look like.
- **main.py** → Builds and runs the whole system from start to finish.

> ***Think of it like this:*** *organising a kitchen before cooking — one drawer for recipes (persona rules), one for utensils (the logic), one for the ingredients list (schemas). Nothing gets built until the kitchen is set up properly.*

Phase 2: Rules & Checklist — Who Is This For, and What Counts as "Good"?

A lesson can only be marked "approved" if it follows strict writing rules and passes a 5-point checklist. This phase defines both.

Who the Learner Is

The lesson is written for a 12th-grade graduate from India who did not study in an English-medium school. That means the writing must follow specific rules:

- **Keep it short:** Sentences should be short — around 12 to 15 words, so they are easy to follow.
- **Use familiar examples:** Explain ideas using everyday Indian examples — like a Kirana store, a school notebook, or a library register — instead of abstract technical comparisons.
- **No unexplained jargon:** Never use a technical term without explaining it in plain words right away.

The 5-Point Checklist (What Gets Graded)

- **1. Accuracy:** Are the facts about the topic correct, with nothing made up?
- **2. Accessibility:** Are the sentences short and easy to read?
- **3. Analogy:** Is there a relatable, real-world comparison to explain the idea?
- **4. Zero Jargon:** Is every technical word explained the moment it's used?
- **5. Structure & Flow:** Does the lesson follow a clear structure: What it is → Why it matters → How it works → Summary?

A small technical detail worth knowing: instead of nesting all these checks inside one another (a "tree" shape), the checklist is kept flat — each check has its own pass/fail flag, its own reason, and its own suggested fix. This avoids errors that happen when AI models are asked to fill in overly complicated formats.

> ***Think of it like this:*** *a report card with 5 separate subjects, each with its own pass/fail mark and teacher's comment — instead of one confusing combined grade.*

Phase 3: Write → Check → Rewrite — How the System Actually Runs

This is the heart of the project: a loop that keeps improving the lesson automatically, without a human stepping in.

The Loop, Step by Step

- **Step 1 — Write:** The system writes a first draft of the lesson based on the topic and the persona rules.

- **Step 2 — Check:** A separate, stricter AI call grades the draft against all 5 checklist points and explains exactly what failed and why.
- **Step 3a — Approved:** If everything passed, the lesson is approved and the process stops here.
- **Step 3b — Rejected:** If something failed and fewer than 3 attempts have been used, the system logs what went wrong, adds 1 to the retry counter, and sends the lesson back to Step 1 — this time with the grading feedback included, so it can actually fix the mistakes.
- **Step 3c — Out of Retries:** If 3 attempts have already been used and it still hasn't passed, the system stops anyway, so it never loops forever.

Why Two Separate AI Calls?

The "writer" and the "checker" are deliberately kept separate, and even run with different settings. The writer is allowed some creative freedom, so its wording varies a little between attempts. The checker is set to be as strict and consistent as possible, so every grading decision is fair and repeatable — it doesn't change its mind randomly.

> ***Think of it like this:*** *having one person write an assignment and a completely different, no-nonsense examiner mark it — rather than letting the same person mark their own homework.*

Phase 4: Memory & Testing — Does It Actually Learn From Its Mistakes?

How the System Remembers Within a Run

While one lesson is being written and graded, the system keeps a running memory of everything that happened:

- **Feedback history:** A running list of every piece of feedback from failed attempts — this gets fed straight back into the next draft, so the writer knows exactly what to fix.
- **Rejection log (audit trail):** A full record of every rejected attempt: what was written, which checks failed, and what fix was suggested. This is useful for reviewing exactly how the lesson improved over time.

How the System Was Tested

To prove the checker actually works (and isn't just rubber-stamping everything), broken lessons were written on purpose, containing:

  - Using technical terms without explaining them.
  - Writing sentences that were far too long.
  - Explaining the topic with abstract ideas instead of relatable examples.

The checker caught every single one of these planted mistakes and explained exactly why each one failed. Then, on the very next attempt, the writer used that feedback to fix the sentence lengths, explain the jargon, and bring back relatable examples — and the lesson passed.

> ***Think of it like this:*** *planting deliberate mistakes in a student's essay to make sure the teacher is actually reading it carefully, not just skimming and approving everything.*

Phase 5: Final Result & Takeaways — What Came Out of This, and What Was Learned

What Was Actually Delivered

- **A GitHub repository:** Clean, organised code, along with a README file explaining how everything is set up and how it works.

- **A finished lesson:** A finished, fully-approved beginner lesson explaining RAG (Retrieval-Augmented Generation) in simple language.
- **A demo video:** A recorded walkthrough covering the code structure, a live run of the system, and a demonstration of the checker catching a deliberately broken lesson and fixing it.

The 3 Big Lessons Learned

- **1. Keep AI output formats simple:** When asking an AI model for structured output, keeping the checklist "flat" (simple pass/fail fields) instead of deeply nested works far more reliably than complicated nested formats.
- **2. Separate creativity from judgment:** Letting the writer be a little creative, but keeping the checker completely strict and consistent, produces more trustworthy grading.
- **3. Always set a maximum retry limit:** Capping the rewrite loop at 3 attempts guarantees the system always finishes, and keeps runtime and cost predictable, instead of possibly looping forever.

In short: this project is a self-correcting AI teacher's assistant — it drafts a lesson, grades its own work honestly, fixes what's wrong using its own feedback, and only hands over a lesson once it genuinely meets the bar, or clearly stops trying after 3 honest attempts.